QML Lecture · 2일차

실습 설명 자료

segmentation 전처리 → 양자 회로 → QGAN 학습

PART 0

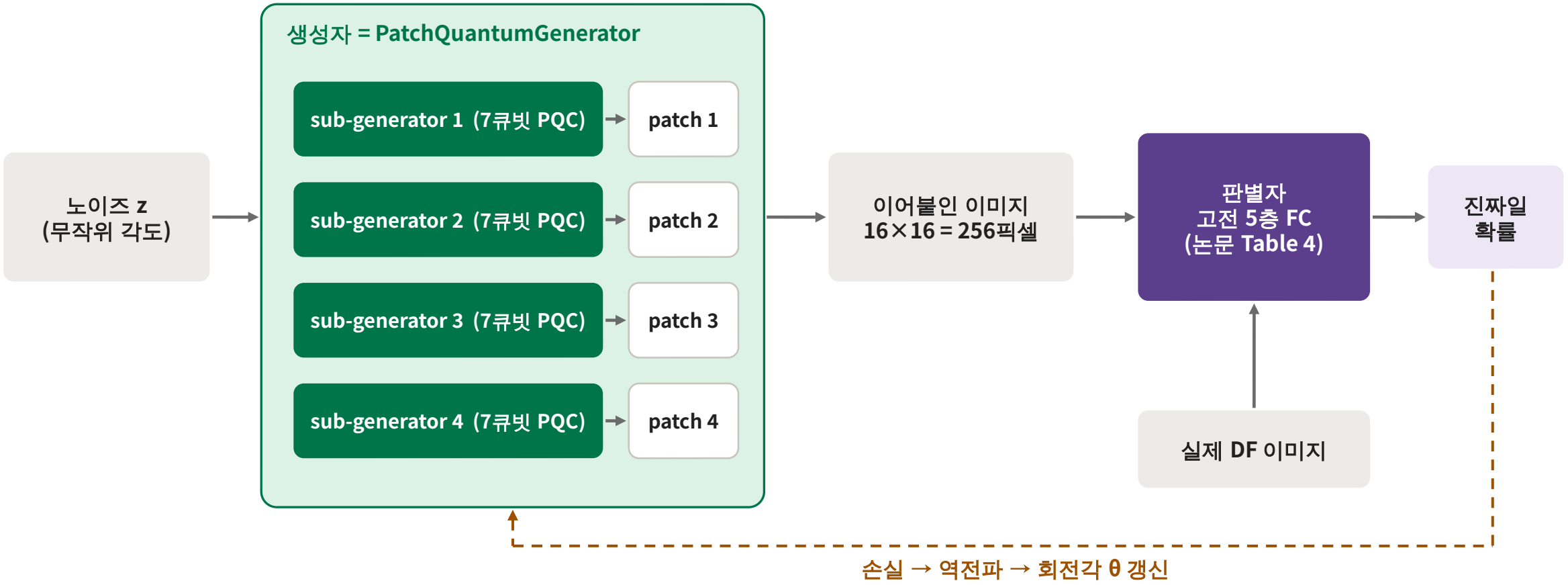
1일차 복습

오늘 구현하는 QGAN의 구조와 논문 대비 축소 범위

오늘 구현하는 것	논문 ① QGAN의 patch 방식 생성자 - sub-generator 4개 + 고전 판별자
입력 데이터	HAM10000 DF 115장 → segmentation → 16×16 그레이스케일
오늘의 결과물	직접 학습한 생성자 checkpoint + 생성품질 지표 4종 표 (3일차 비교 기준)

QGAN 구조 : 생성자만 양자회로

patch 방식: sub-generator마다 이미지 조각(patch)을 하나씩 만들고 이어붙임



논문과 오늘 실습의 차이

회로의 형태(RY encoding + RY·CZ 변분층 + post-selection)는 같고, 규모와 학습량을 줄임

항목	논문 (Andra et al.)	2일차 실습	이유
sub-generator	16개	4개	2·4·8·16개 실측 중 품질 최고
큐빗 (sub-generator당)	데이터 7 + ancilla 1	데이터 6 + ancilla 1	$256 = 4 \times 2^6$
회로 depth	10	10	논문과 동일
판별자	5층 FC (Table 4)	5층 FC (Table 4)	논문과 동일
학습량	2,000 epoch (1 ~ 1.5시간)	400 step (Colab 약 35분)	수업 시간 안에 완료
실행 환경	GPU	Colab CPU, exact 시뮬레이션	설치·계정 불필요

논문 설정인 sub-generator 16개로 학습하면 판별자가 이겨 loss_d가 0.003까지 떨어짐. 용량을 늘려도 품질이 오르지 않았음.

논문 값: Andra et al. 프리프린트 Table 3 · 실습 값: README sub-generator 실험

PART 1 · 노트북 §2 ~ §3

데이터와 segmentation 전처리

HAM10000에서 DF·NV를 고르고, 병변만 남겨 16×16으로 줄이는 과정

노트북 범위	§2 EDA, §3 segmentation 전처리
다루는 코드	segment_fcm_kmeans · resize_for_encoding · preprocess_class
논문 근거	Andra et al. Section 3.1 (K-means + Fuzzy C-means), Table 10

범위 축소 : DF vs NV

7개 클래스 중 가장 적은 DF를 증강 대상으로, 가장 많은 NV를 비교 대상으로 둠

클래스	이미지 수	비율	실습에서의 역할
DF (피부섬유종)	**115**	1.1%	증강 대상 - 생성자 학습 데이터
NV (모반)	6,705	66.9%	비교 대상 - 지표 ② 기준 (345장 사용)
전체 7개 클래스	10,015	100%	DF : NV = 1 : 58

```
metadata = pd.read_csv(f"{DATA_DIR}/HAM10000_metadata.csv")
counts = metadata["dx"].value_counts()

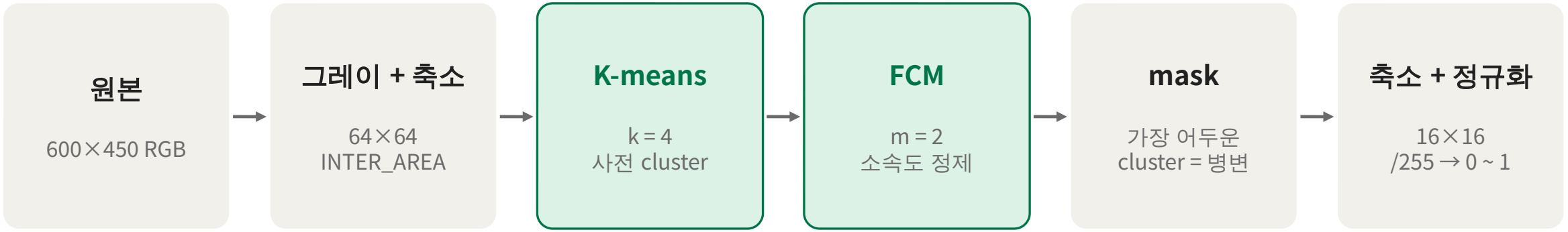
# df     115   <- 증강 대상
# nv    6705   <- 비교 대상
```

- metadata는 image_id와 진단명(dx)을 연결하는 표임
- 이미지 파일은 part_1·part_2 두 폴더에 나뉘어 있어 find_image_path가 양쪽을 확인함

HAM10000 메타데이터 (Tschandl et al., Scientific Data 5, 180161, 2018)

전처리 pipeline

원본 → 병변만 남긴 64×64 → 16×16, 0 ~ 1. 저장소 자산(df_preprocessed.npy)도 같은 조건으로 만들어짐



논문 Section 3.1의 K-means + Fuzzy C-means hybrid segmentation

병변만 남기는 이유

논문 Table 10: raw 이미지로 증강하면 분류 성능이 오히려 떨어지고, segmentation을 거친 이미지에서만 개선됨

64×64를 거치는 이유

FCM 비용이 픽셀 수에 비례함. 원본 27만 픽셀 대신 4,096픽셀로 clustering해 1장당 수 초로 줄임

segment_fcm_kmeans 코드

밝기 1채널만 써서 4개 cluster로 나누고, 가장 어두운 cluster를 남김

```
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
gray = cv2.resize(gray, (64, 64), interpolation=cv2.INTER_AREA)
pixels = gray.reshape(-1, 1).astype(np.float32)      # 1

km = KMeans(n_clusters=4, n_init=4, random_state=0)  # 2
km = km.fit(pixels)
dist = np.abs(pixels - km.cluster_centers_.reshape(1, -1))
u_init = 1.0 / (dist + 1e-6) ** 2                    # 3
u_init = (u_init / u_init.sum(axis=1, keepdims=True)).T

cntr, u, *_ = fuzz.cluster.cmeans(pixels.T, c=4, m=2.0,
                error=1e-4, maxiter=100, init=u_init)
membership = np.argmax(u, axis=0)                    # 4
lesion = np.argmin(cntr.flatten())
mask = ((membership == lesion).reshape(64, 64) * 255).astype(np.uint8)
return cv2.bitwise_and(gray, gray, mask=mask)        # 5
```

① 밝기만 사용

위치 정보 없이 (픽셀 수, 1)로 clustering

② K-means 사전 cluster

k-means++ 초기화 4회 중 최선 채택

③ FCM 초기 소속도

중심 거리의 역제곱 - 수렴이 빠르고 재현 가능

④ 병변 cluster 선택

중심이 가장 어두운 cluster = 병변

⑤ mask 적용

병변은 원래 밝기 유지, 배경은 0

K-means와 Fuzzy C-means

K-means는 픽셀마다 cluster 하나, FCM은 cluster별 소속도를 나눠 가짐

	K-means	Fuzzy C-means (FCM)
배정 방식	hard - 픽셀마다 cluster 1개	soft - 소속도 u (픽셀별 합 = 1)
중심 갱신	배정된 픽셀의 평균	소속도^m 가중 평균
주요 파라미터	n_clusters, n_init	c, m (fuzziness), error, maxiter, init
이 코드에서의 역할	FCM 초기값을 만드는 사전 cluster	소속도 정제 → argmax로 최종 라벨
출력	cluster_centers_, labels_	cntr, u, u0, d, jm, p, fpc

m (fuzziness)

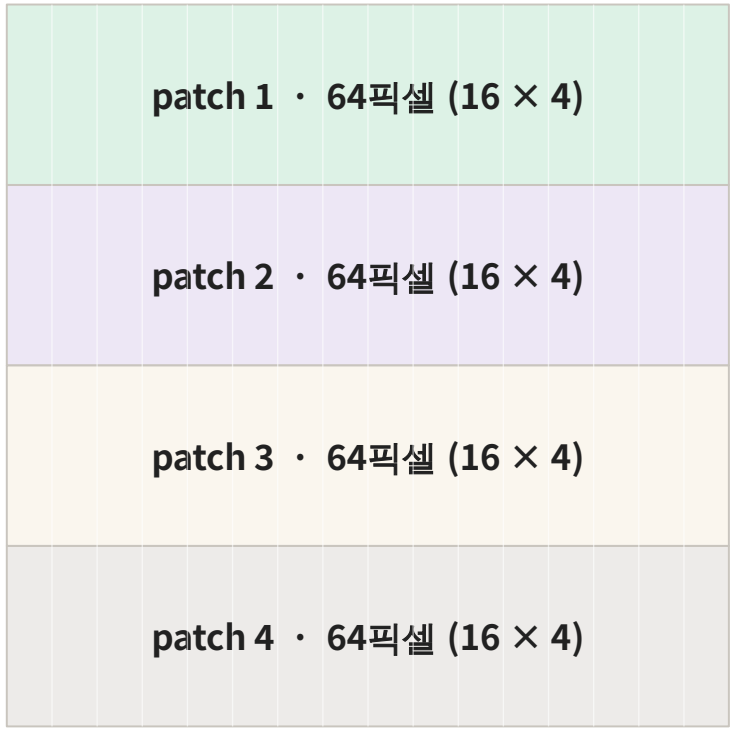
- m → 1: K-means처럼 딱 잘라 나뉨
- m이 클수록 소속이 여러 cluster로 퍼짐
- 보통 2를 씀 (이 코드도 2)

fpc (0 ~ 1)

분할 선명도. 1에 가까울수록 명확하고, 1/c(여기서는 0.25)에 가까울수록 모호함

16×16 해상도와 축소 방법

픽셀 수가 큐빗 수를 정함. 256픽셀 = sub-generator 4개 × 2^6



16 × 16 = 256픽셀

큐빗 수 계산

256 ÷ 4 = 64픽셀/patch → $\log_2 64 = 6$ 데이터 큐빗 → + ancilla 1 = 7큐빗

보간법	방식	16×16 축소에서
INTER_AREA	출력 픽셀이 덮는 영역의 평균	사용 - 저장소 자산과 같은 조건
INTER_LINEAR	주변 2×2 선형 보간 (기본값)	큰 축소에서 앨리어싱
INTER_NEAREST	가장 가까운 픽셀 1개 복사	마스크용, 밝기 정보 손실

PART 2 · 노트북 §4 ~ §5

양자 회로 : sub-generator 하나

노이즈 encoding → 변분층 반복 → 측정 → post-selection

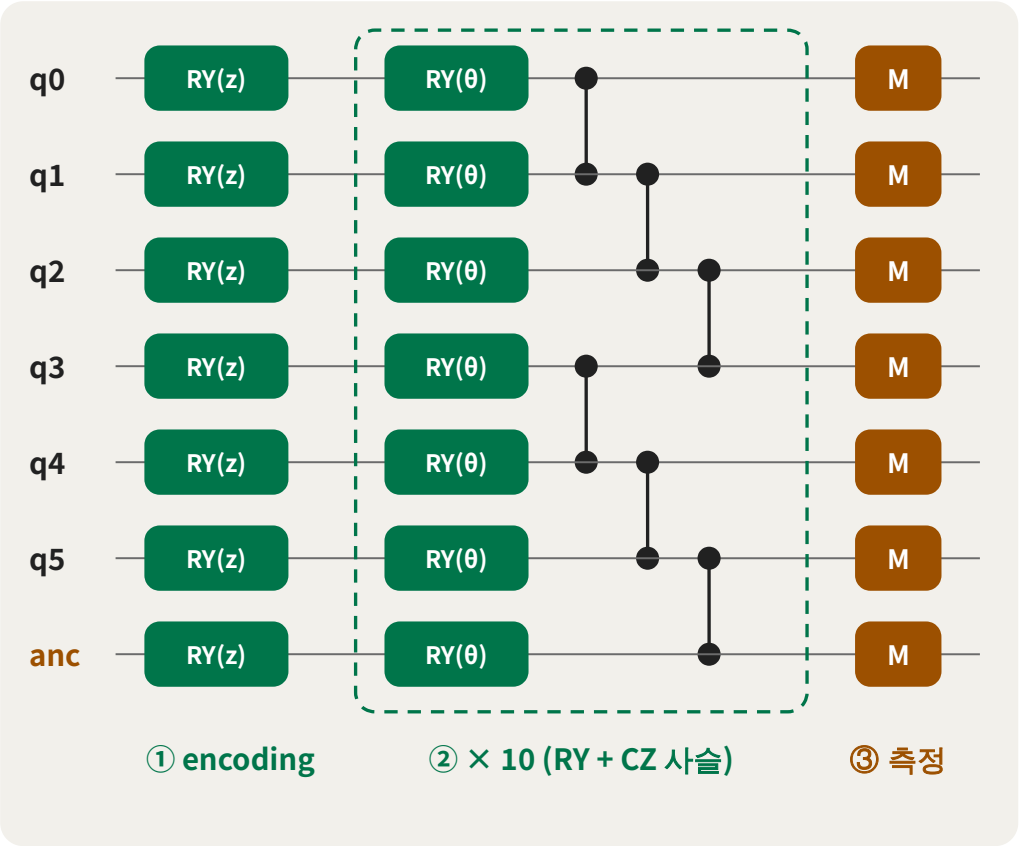
노트북 범위	§4 회로 1개 출력 확인, §5 생성자·판별자 정의
다루는 코드	sub_generator_circuit · probs_all[::2] · PatchQuantumGenerator

sub-generator 회로

RY로 노이즈를 넣고, [학습 RY + 인접 CZ]를 10번 반복한 뒤 7큐빗 전체를 측정

```
@qml.qnode(dev)
def sub_generator_circuit(noise, weights):
    for i in range(N_QUBITS):           # 1 encoding
        qml.RY(noise[i], wires=i)
    for layer in range(Q_DEPTH):        # 2 x 10
        for i in range(N_QUBITS):
            qml.RY(weights[layer, i], wires=i)
        for i in range(N_QUBITS - 1):
            qml.CZ(wires=[i, i + 1])
    return qml.probs(wires=range(N_QUBITS))  # 3
```

weights 모양 (10, 7) - sub-generator 하나당 학습 파라미터 70개



ancilla qubit과 post-selection

128개 확률 중 ancilla qubit = 0인 64개만 patch로 씀. 코드로는 probs_all[::2]

인덱스	q0 … q5 (데이터)	anc	사용
0	000000	0	**patch 픽셀 0**
1	000000	1	버림
2	000001	0	**patch 픽셀 1**
3	000001	1	버림
…	…	…	…
126	111111	0	**patch 픽셀 63**
127	111111	1	버림

```
probs_all = sub_generator_circuit(noise, weights)
output = probs_all[::2]   # even index = anc 0
```

ancilla 없이 측정하면

- 64개 확률의 합 = 항상 1
- 배경 patch(합 ≈ 0)와 병변 patch(합이 큼)를 구분해 표현할 수 없음

ancilla = 0만 선택하면

- 64개 확률의 합 = 0 ~ 1 사이에서 자유로움
- patch별 밝기 총합도 학습 대상이 됨
- 회로는 그대로, 측정·해석만 바꾼 것

PatchQuantumGenerator : patch 4개를 이어 한 장으로

sub-generator별로 batch 크기만큼 회로를 실행하고, patch를 가로로 이어붙임

```
class PatchQuantumGenerator(nn.Module):
    def __init__(self, n_generators, n_data_qubits, ...):
        q_dev = qml.device("default.qubit", ...)
        @qml.qnode(q_dev, interface="torch",
                   diff_method="backprop")
        def circuit(noise, weights): ...   # same as §4
        self.q_params = nn.ParameterList([
            nn.Parameter(torch.rand(q_depth, n_qubits) * pi)
            for _ in range(n_generators)])

    def forward(self, batch_size):
        patches = []
        for g in range(self.n_generators):      # 4
            rows = [self.circuit(torch.rand(7) * pi,
                     self.q_params[g])[::2]
                    for _ in range(batch_size)] # 8
            patches.append(torch.stack(rows))
        return torch.cat(patches, dim=1)   # (batch, 256)
```

gen(8) 한 번의 실행

sub-gen 1	× 8 샘플	(8, 64)
sub-gen 2	× 8 샘플	(8, 64)
sub-gen 3	× 8 샘플	(8, 64)
sub-gen 4	× 8 샘플	(8, 64)

torch.cat(dim=1) → (8, 256)

회로 실행 4 × 8 = 32번

파라미터 수 : 생성자 280개, 판별자 19,201개

약 70배 차이가 나서 학습률을 다르게 둠 (생성자 0.05, 판별자 0.001)

구성	모양	파라미터
생성자: 회전각 RY(θ)	**sub-generator 4 × depth 10 × 큐빗 7**	**280**
판별자 Linear 1	256 → 64	16,448
판별자 Linear 2	64 → 32	2,080
판별자 Linear 3	32 → 16	528
판별자 Linear 4	16 → 8	136
판별자 Linear 5 + Sigmoid	8 → 1	9
판별자 합계		**19,201**

```
opt_g: lr=0.05
opt_d: lr=0.001
```

판별자가 약 70배 크고 빨리 배우므로, 생성자 학습률을 크게, 판별자는 작게 둬 균형을 맞춤.

PART 3 · 노트북 §5

QGAN 학습

학습 루프, 손실 읽는 법, 계산 비용과 한계

노트북 범위	§5 학습 셀 (400 step)
다루는 코드	학습 루프 (판별자 step · 생성자 step) · BCE 손실
소요 시간	Colab CPU 약 35분, 진행 로그는 20 step마다

학습 루프 : 한 step에 두 번 갱신

판별자를 먼저 갱신하고, 같은 step에서 생성자를 갱신함

```
for step in range(STEPS):                    # 400
    batch = real_flat[torch.randint(0, 115, (8,))]

    # discriminator: real -> 1, fake -> 0
    opt_d.zero_grad()
    loss_d = bce(disc(batch), ones) \
           + bce(disc(gen(8).detach()), zeros)
    loss_d.backward(); opt_d.step()

    # generator: make disc say 1 for fakes
    opt_g.zero_grad()
    loss_g = bce(disc(gen(8)), ones)
    loss_g.backward(); opt_g.step()
```

① 판별자 step

실제 → 1, 생성 → 0. detach로 생성자 쪽 기울기는 계산하지 않음

② 생성자 step

생성 이미지에 라벨 1을 붙여 손실 계산. 판별자를 속이는 방향으로 θ 갱신

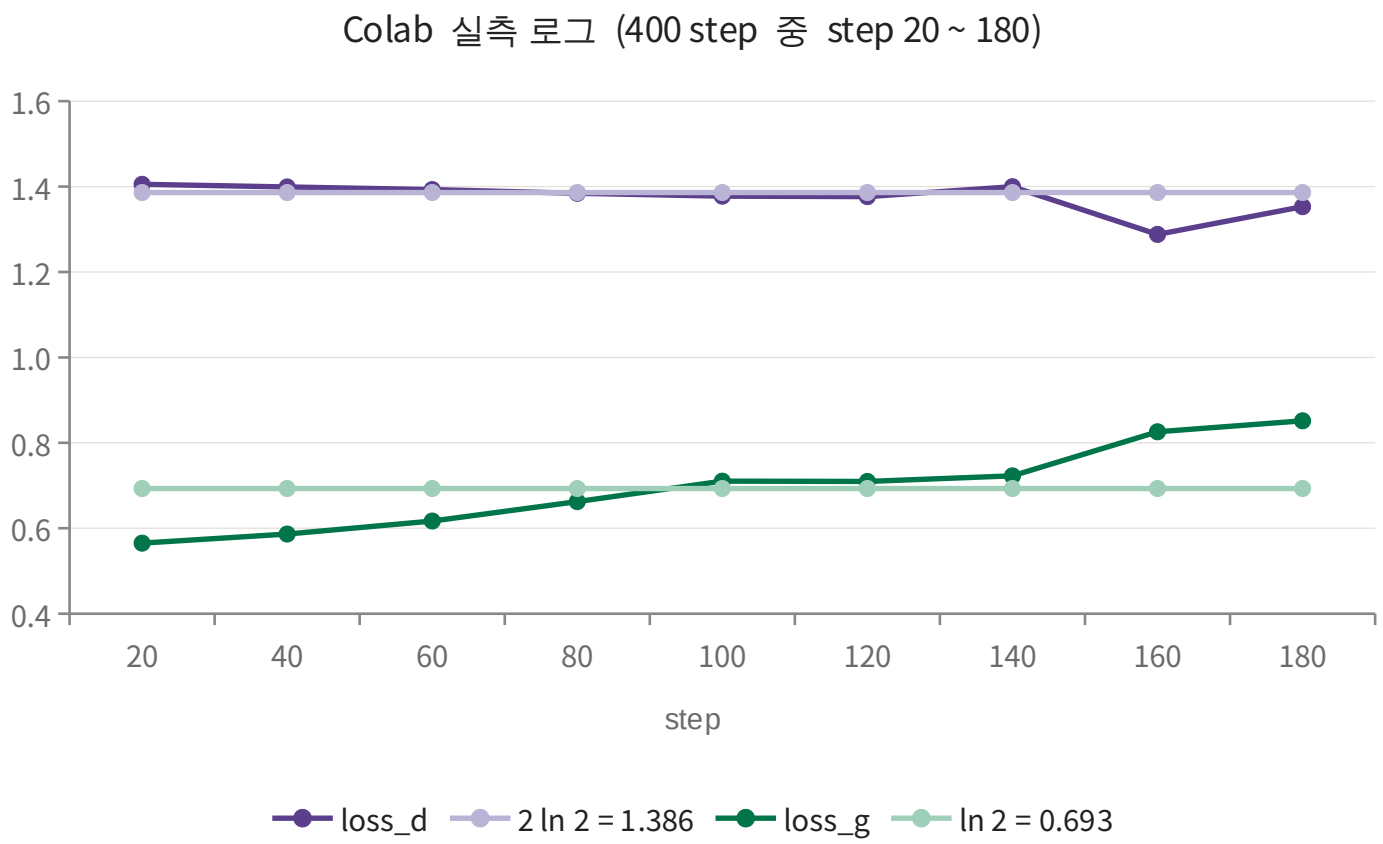
③ gen() 호출 2번

판별자용 1번 + 생성자용 1번. step당 회로 64번 실행

PART 3 · 진행 로그 읽기

손실 기준값 : 1.386과 0.693

판별자가 진짜·가짜를 구분하지 못할 때(D = 0.5)의 BCE 값이 기준선



값	의미
loss_d ≈ 1.386	구분 불가 - 경쟁 유지
loss_d → 0	판별자 완승, 생성자 신호 소실
loss_g ≈ 0.693	생성 이미지를 반반으로 판정
loss_g = 0.85	D(G(z)) ≈ e^−0.85 ≈ 0.43

GAN 손실은 단조 감소하지 않음. 손실이 균형처럼 보여도 품질은 낮을 수 있어 §6 지표로 따로 평가함.

계산 비용 : 학습에 35분이 걸리는 이유

회로 하나는 작지만($2^7 = 128$차원) 실행 횟수가 많음

회로 실행 수 = sub-generator 4 × batch 8 × step당 gen() 2회 × 400 step = 25,600회

회로 하나는 7큐빗(상태벡터 128차원)으로 작지만, 이 횟수만큼 forward와 backprop을 반복함.

미분 방식	어디서	샘플 하나의 생성자 기울기 비용	사용처
backprop	시뮬레이터만	forward 1번 + 역전파 (상태벡터 직접 미분)	2일차 QGAN (default.qubit)
adjoint	시뮬레이터만	forward 1번 + 역방향 1번, 메모리 적음	3일차 MediQ-GAN (lightning)
parameter-shift	하드웨어 가능	파라미터당 2번 → 280 × 2 = 560번 (× shot 수)	실제 양자 장치

PART 3 · 한계

한계 : 실제와 생성 이미지의 픽셀 합 불일치

정규화 단위가 달라, 판별자가 형태가 아닌 밝기 총합만으로도 구분할 수 있음

	정규화 단위	이미지 한 장의 픽셀 합
실제 이미지 (real_flat)	이미지 전체	항상 1
생성 이미지 (gen(...))	patch 단위 (ancilla = 0 확률)	0 ~ 4 (Colab 실측 약 1.63)

```
real_flat = real_flat / real_flat.sum(dim=1, keepdim=True)   # sum = 1
fake      = gen(8)            # each of 4 patches sums to 0..1
```

수정 방법 (실습 후 과제)

실제 이미지도 patch별로 정규화

patch마다 합 1로 맞춰 생성 쪽과 단위를 맞춤

생성 patch를 최댓값으로 나눔

0 ~ 1 척도로 맞추고 실제 이미지는 그대로 둠 (PennyLane QGAN 튜토리얼 방식)

PART 4 · 노트북 §6

생성과 평가

생성 이미지를 지표 4종으로 평가함. 3일차 MediQ-GAN에도 같은 지표를 씀

노트북 범위	§6 생성 이미지 확인과 지표 4종
다루는 코드	gm.to_unit_scale · mean_image_correlation · nearest_neighbor_distance · sample_diversity

생성품질 지표 4종

16×16 저해상도라 FID 대신 해석 가능한 지표를 씀 (practice/gen_metrics.py)

지표	보는 것	함수	읽는 법
① 픽셀 강도 분포	밝기 분포가 실제와 맞는가	plot_pixel_histogram	척도 변환 전 값도 함께 봄
② 클래스 평균 상관	병변의 전체 형태를 재현했는가	mean_image_correlation	높을수록 좋음. DF > NV여야 함
③ 최근접 실제 거리	학습 데이터를 외웠는가	nearest_neighbor_distance	0에 가까우면 암기
④ 샘플 간 거리	같은 그림만 반복하는가	sample_diversity	0에 가까우면 mode collapse

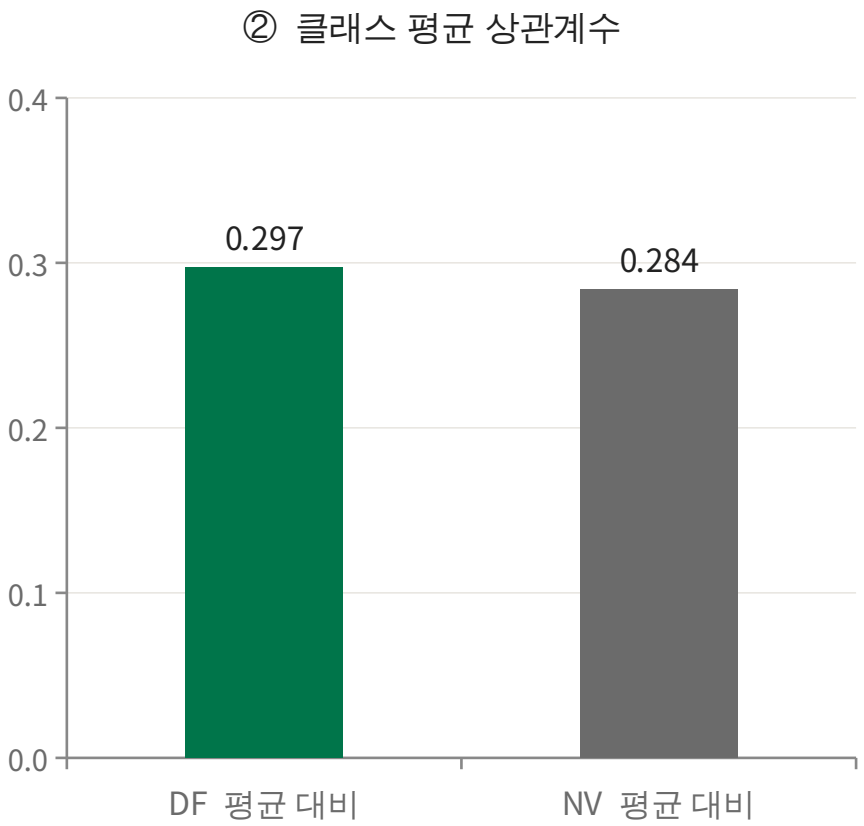
to_unit_scale(x, mode) - 세 척도를 0 ~ 1로 맞춘 뒤 지표를 계산함

mode	대상	변환
prob	QGAN 출력 (patch별 합 0 ~ 1인 확률)	이미지별 min-max
tanh	MediQ-GAN 출력 (-1 ~ 1)	(x + 1) / 2
unit	전처리 데이터 (0 ~ 1)	그대로

PART 4 · 결과 읽기

Colab 실행 결과

형태는 어느 정도 재현했지만 DF와 NV의 상관 차이는 작음. 실행마다 값이 조금씩 다름



지표	결과	해석
② 상관 (DF / NV 평균 대비)	0.297 / 0.284	DF와 NV 차이가 작음
③ 최근접 실제 거리	평균 1.579, 최솟값 0.956	0보다 충분히 커서 암기는 아님
④ 샘플 간 거리	1.731	실제 DF 2.505의 약 70% - mode collapse 아님

DF와 NV 상관이 거의 같음. 생성자가 DF 고유의 특징을 잡았는지, 16×16 그레이스케일에서 두 클래스의 평균 형태가 원래 비슷한지는 이 결과만으로 구분할 수 없음.

Colab에서 노트북을 끝까지 실행한 결과 (400 step, 생성 64장)

정리

2일차 정리

segmentation 전처리부터 QGAN 학습·평가까지

주제	정리
데이터와 전처리	DF 115장을 증강 대상으로 둠. K-means + FCM segmentation으로 병변만 남기고 16×16, 0 ~ 1로 축소 (INTER_AREA)
sub-generator 회로	7큐빗(데이터 6 + ancilla 1), RY encoding 후 [학습 RY + 인접 CZ] × 10. ancilla = 0인 확률 64개가 patch 하나
post-selection	ancilla = 0 항목만 쓰면 patch 밝기 총합이 0 ~ 1에서 자유로워져 배경 patch도 표현 가능
생성자·판별자	생성자 280개(양자 회전각) vs 판별자 19,201개. 학습률 0.05 / 0.001로 균형을 맞춤
학습과 손실	step마다 판별자 → 생성자 순서로 갱신. 기준값 loss_d 1.386, loss_g 0.693, loss_d → 0이면 판별자 완승
계산 비용	400 step에 회로 25,600회. backprop은 시뮬레이터 전용, 실제 장치는 parameter-shift(파라미터당 2회 × shot)
평가와 한계	지표 4종을 함께 봄. 실제(합 1)와 생성(patch별 합) 이미지의 픽셀 합 불일치, 16×16, shot noise 없음